58. Перечисления

Язык C++ позволяет программистам создавать свои собственные пользовательские типы данных.

Оглавление:

1. Перечисляемые типы
2. Имена перечислений
3. Значения перечислений
4. Обработка перечислений
5. Вывод перечислений
6. Выделение памяти для перечислений
7. Польза от перечислений
8. Тест

Перечисляемые типы

Перечисление (или **перечисляемый тип**) — это тип данных, где любое значение (или **перечислимые**) определяется как **символьная константа**. Объявить перечисление можно с помощью **ключевого слова enum**. Например:

```
enum Colors
{
    // Все перечисления перечисления с или маленькой буквами этого типа разные.
    // Перечисления можно назначить разным значениям (см. пример с enum2)
    COLOR_RED,
    COLOR_ORANGE,
    COLOR_GRAY,
    COLOR_GREEN,
    COLOR_PINK,
    COLOR_BROWN,
    COLOR_BLUE,
    COLOR_PURPLE,
};

// Объявление переменной перечисленного типа Colors
Colors paint = COLOR_BROWN;
Colors hair = COLOR_GRAY;
```

Объявление перечислений не требует выделения памяти. Только когда переменная перечисленного типа определена (например, как переменная paint в примере, приведенном выше), только тогда выделяется память для этой переменной.

Обратите внимание, каждый перечисленный объект оканчивается запятой, а сама перечисление заканчивается точкой с запятой.

Примечание: До C++11, конечная запятая после последнего перечисления (как после COLOR_PURPLE в примере, приведенном выше) не разрешается (хотя многие компиляторы им все равно принимают). Однако начиная с C++11 конечная запятая разрешена.

Имена перечислений

Идентификаторы перечислений часто начинаются с заглавной буквы, а имена перечислений вообще состоят только из заглавных букв. Поскольку перечисления вместе с перечислениями находятся в едином **пространстве имен**, то имена перечислений не могут повторяться в разных перечислениях:

```
enum Colors
{
    YELLOW,
    BLACK, // BLACK относится к глобальным пространству имен
    PINK
};

enum Feelings
{
    SAD,
    ANGRY,
    BLACK // ошибка: ранее, так как BLACK уже использовался в enum Colors
};
```

Распространяя добавление названия перечисления в качестве префикса к перечислениям, например: ANIMAL_ и COLOR_, как для предотвращения **конфликтов имен**, так и в целях комментирования кода.

Значения перечислений

Каждому перечислению автоматически присваивается целочисленное значение в зависимости от его позиции в списке перечисления. По умолчанию, первому перечислению присваивается целое число 0, а каждому следующему — на единицу больше, чем предыдущему:

```
// Перечисления
enum Colors
{
    COLOR_YELLOW, // присваивается 0
    COLOR_WHITE, // присваивается 1
    COLOR_ORANGE, // присваивается 2
    COLOR_GREEN, // присваивается 3
    COLOR_BLUE, // присваивается 4
    COLOR_GRAY, // присваивается 5
    COLOR_PURPLE, // присваивается 6
    COLOR_BROWN // присваивается 7
};

int main()
{
    Colors paint(COLOR_RED);
    std::cout << paint;

    return 0;
}
```

Результат выполнения программы:

```
4
```

Можно и самому определить значения перечислений. Они могут быть как положительными, так и отрицательными, или вообще иметь аналогичные другим перечислениям значения. Любые не определенные вами перечислителям, будут иметь значение на единицу больше, чем значение предыдущего перечислителя. Например:

```
// Определение новых перечислений под Animals
enum Animals
{
    ANIMAL_CAT = 4,
    ANIMAL_DOG, // присваивается 5
    ANIMAL_PIG, // присваивается 6
    ANIMAL_HORSE = 9,
    ANIMAL_GOAT, // присваивается 10
    ANIMAL_COW // присваивается 11
};

// Определение новых перечислений под Mammals
enum Mammals
{
    MAMMAL_CAT = 4,
    MAMMAL_DOG, // присваивается 5
    MAMMAL_PIG, // присваивается 6
    MAMMAL_HORSE = 9,
    MAMMAL_GOAT, // присваивается 10
    MAMMAL_COW // присваивается 11
};

int main()
{
    int mpet = MAMMAL_PIG;
    std::cout << ANIMAL_HORSE << " " << mpet << " " << MAMMAL_COW << " " << MAMMAL_CAT << " " << MAMMAL_DOG << " " << MAMMAL_PIG << " " << MAMMAL_HORSE << " " << MAMMAL_GOAT
```